Parallel Optimization of JPEG Compression Algorithm

Kun Yao Lin
311551170
IOC, NYCU
Hsinchu, Taiwan
enk857@gmail.com

Pei Chi Chen
312553008
ILE, NYCU
Hsinchu, Taiwan
a0909205568@gmail.com

Shun Lin
312553027
ILE, NYCU
Hsinchu, Taiwan
linsapphire@gmail.com

1 ABSTRACT

This study examines the performance of PThread, OpenMP, and CUDA in the context of JPEG compression across different image resolutions. By comparing speedup results, we analyze the impact of data transfer overhead, thread management, and computational efficiency on parallel performance. The findings provide insights into the behavior of CPU-based and GPU-based parallelization methods when applied to JPEG compression workloads.

2 INTRODUCTION

JPEG compression is one of the most widely used image compression algorithms, enabling efficient storage and transmission of digital images. However, with the rise of high-definition imagery and the demand for real-time processing, speeding up JPEG compression has become crucial. This project aims to optimize the performance of JPEG compression using parallel programming techniques, specifically focusing on parallelizing computationally expensive steps such as the Discrete Cosine Transform (DCT) and quantization. By leveraging multi-core processors through OpenMP and CUDA for GPU acceleration, we can divide the image into independent blocks and process them in parallel. This will enhance key operations like color space conversion, DCT, quantization, and frequency table building, significantly reducing processing time while maintaining image quality.

3 STATEMENT OF PROBLEM

In this project, we aim to optimize the JPEG compression algorithm using parallel programming techniques. JPEG compression consists of several computational steps, which we will break down and analyze to identify potential areas for parallelization. The key stages of the JPEG compression algorithm are as follows: **Color Space Conversion, Discrete Cosine Transform (DCT), Quantization, ZigZag Scanning, Run-Length Encoding (RLE), and Huffman Encoding**, as shown in the Figure 1 below.

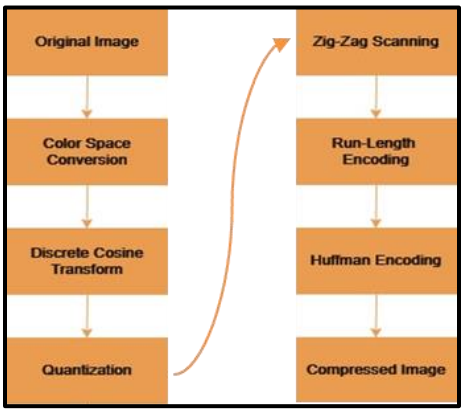


Figure 1: The flow chart of JPEG compression.

Below, we provide a detailed explanation of each step and analyze which parts can be parallelized for performance improvement.

3.1 Color Space Conversion

In JPEG compression, the input image is first converted from the RGB color space to the YCbCr color space. This transformation is applied to each pixel independently using the following formulas:

$$\mathrm{Y} = 0.299\mathrm{R} + 0.587\mathrm{G} + 0.114\mathrm{B}$$
$$Cb = 128 - 0.168736R - 0.331264G + 0.5B$$
$$Cr = 128 + 0.5R - 0.418688G - 0.081312B$$

3.2 Discrete Cosine Transform (DCT)

The DCT is applied to each 8×8 block of the Y, Cb, and Cr components to convert spatial image data into frequency data. The DCT for each block can be expressed as:

$$F(u,v) = \frac{1}{4}\sum_{x=0}^{7}\sum_{y=0}^{7} f(x,y)\cos\left[\frac{(2x+1)u\pi}{16}\right]\cos\left[\frac{(2y+1)v\pi}{16}\right]$$

Where $f(x,y)$ is the pixel value in the block, and $F(u,v)$ is the DCT coefficient.

3.3 Quantization

In the quantization stage, the DCT coefficients are divided by a quantization matrix to reduce their precision and discard less important data. The quantized value $Q(u,v)$ can be expressed as:

$$Q(u,v) = round\left(\frac{F(u,v)}{Q_{table}(u,v)}\right)$$

Where $Q_{table}(u, v)$ is the corresponding entry in the quantization table.

3.4 ZigZag Scanning

After quantization, the 8×8 block of DCT coefficients is converted into a one-dimensional array using ZigZag scanning. This step reorders the coefficients in a way that clusters low-frequency components (which are more likely to be significant) before high-frequency ones (which are often zeros). The ZigZag scan follows this pattern:

0	1	5	6	14	15	27	28
2	4	7	13	16	26	29	42
3	8	12	17	25	30	41	43
9	11	18	24	31	40	44	53
10	19	23	32	39	45	52	54
20	22	33	38	46	51	55	60
21	34	37	47	50	56	59	61
35	36	48	49	57	58	62	63

3.5 Run-Length Encoding (RLE)

Once the coefficients are reordered using ZigZag, Run-Length Encoding (RLE) is applied to compress sequences of consecutive zeros in the frequency data. This reduces the size of the data by representing multiple zeros with a single count and value. For example, the sequence $[15, 0, 0, 0, 6, 0, 0, 0]$ would be encoded as $[(15), (0,3), (6), (0,3)]$.

3.6 Huffman Encoding

Huffman encoding relies on building a frequency table for unique coefficients. Each coefficient is assigned a variable-length binary code based on its frequency, with more common symbols receiving shorter codes. The process involves analyzing run-length encoded data to build the frequency table and then constructing a binary tree for encoding

3.7 Why We Chose This Problem

The JPEG compression algorithm is applied to image processing and requires calculations on each pixel, resulting in relatively long execution times. However, a careful analysis of the algorithm's various steps reveals that many of them involve traversing all pixels in the image, with each pixel's calculations being independent. For example, steps like the Discrete Cosine Transform (DCT), and Quantization can all be executed using loops for traversal. This indicates that we can parallelize these steps to accelerate the execution of the algorithm. Therefore, the next chapter will discuss our plans for parallelizing these steps.

4 PROPOSED SOLUTION

This section outlines key steps in our approach, emphasizing their potential for parallelization. We analyze which processes can be performed concurrently to improve efficiency and explain the limitations for those requiring sequential execution.

4.1 Color Space Conversion

Since this process is performed independently for each pixel, it can be easily parallelized. By dividing the image into smaller chunks, we can assign multiple threads to handle the conversion of different regions of the image simultaneously.

4.2 Discrete Cosine Transform (DCT)

Since the DCT is computed on a block-by-block basis, each 8×8 block can be processed independently, making it highly parallelizable. Threads can be assigned to perform DCT on multiple blocks concurrently, reducing overall computation time.

4.3 Quantization

Like the DCT, quantization is performed independently for each 8×8 block, making it amenable to parallelization. Multiple threads can handle quantization for different blocks in parallel.

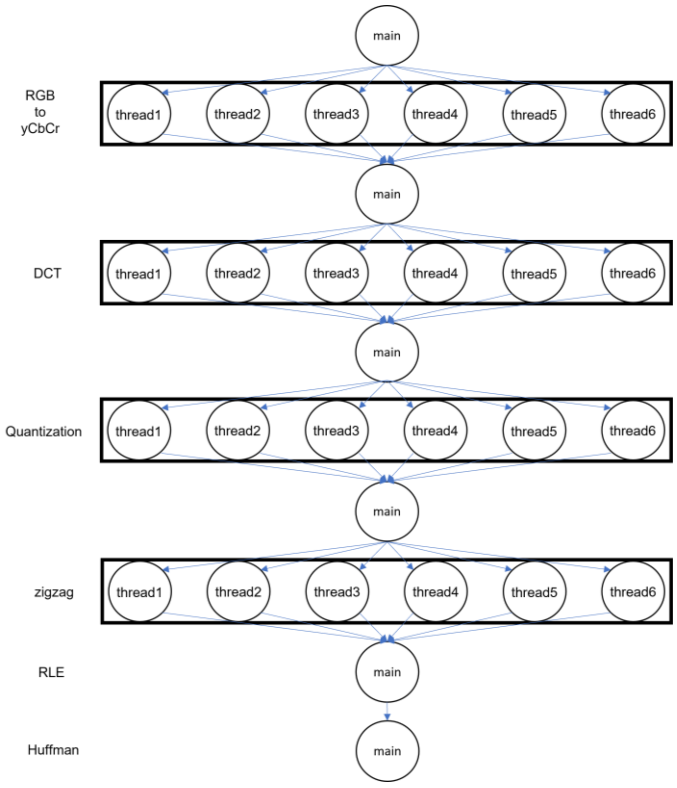


Figure 2: Parallel Strategies and Flow Chart of JPEG compression Parallelism.

4.4 ZigZag Scanning

ZigZag scanning is the process of converting the DCT coefficients in an 8×8 block into a one-dimensional sequence. Since this scanning process must be performed in a specific order, it cannot be parallelized. The data within each block must be sequentially converted according to a fixed path, meaning each block must be scanned step by step, and multiple data points cannot be processed simultaneously.

4.5 Run-Length Encoding (RLE)

After ZigZag scanning, the scanned results undergo RLE encoding. The core of RLE is compressing consecutive occurrences of the same number into a value and count pair (e.g., if the number 0 appears 5 times consecutively, it is recorded as (0, 5)). This encoding process is performed independently for each block, so multiple threads can handle RLE encoding for different blocks in parallel, improving efficiency.

4.6 Huffman Encoding

Huffman encoding is used to further compress the data after RLE. By constructing a Huffman tree, shorter codes are assigned to frequent symbols, while longer codes are assigned to less frequent symbols. Huffman encoding is typically performed at the whole image level, so unlike previous steps, its parallelization is limited.

5 EXPERIMENTAL METHODOLOGY

We used C++ for its efficiency and flexibility, leveraging tools like the Standard Template Library (STL) and frameworks such as OpenMP and CUDA to support parallelization.

The input comprised three image resolutions: 200 × 200, 512 × 512, and 1920 × 1280. These sizes were chosen to test scalability and performance under varied workloads.

Two parallelization approaches were explored:

i. Entire Image Parallelization: The entire image is processed in parallel, with threads dividing the workload evenly. This method maximizes resource utilization for larger images.

ii. Step-Wise Parallelization: The image processing pipeline is divided into stages (e.g., pre-processing, feature extraction), and each stage is parallelized independently. This allows better control for complex workflows.

Our experiments were conducted on servers provided as part of the course environment.

6 EXPERIMENTAL RESULTS

6.1 Pthread & OpenMP

As shown in Figure 3, increasing the number of threads from 1 to 6 significantly reduces execution time, demonstrating the effectiveness of parallelization. However, performance gains slow after 4 threads, likely due to resource contention and memory bandwidth limitations. PThread and OpenMP perform similarly, as both manage threads effectively, with OpenMP reducing programming overhead through higher-level abstractions.

In Figure 4, PThread's execution time increases with more threads due to synchronization overhead, while OpenMP remains stable and performs better in fine-grained parallelization. Overall, OpenMP is more suitable for block-level parallelization, while PThread suffers from higher overhead.

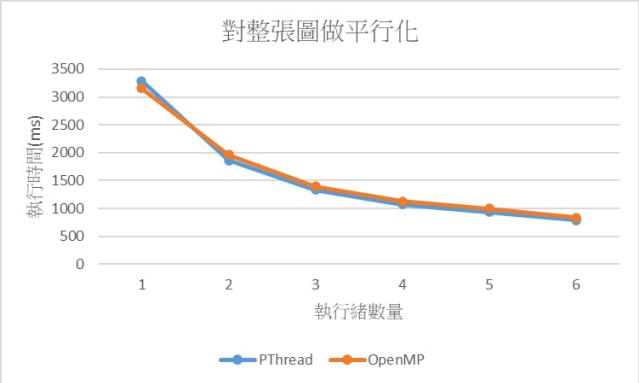


Figure 3: Entire Image Parallelization Using Pthread and OpenMP.

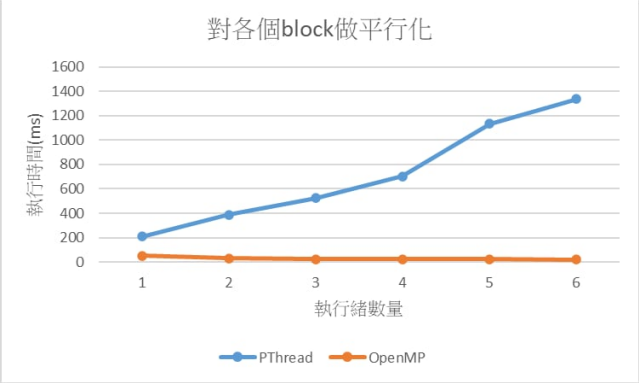
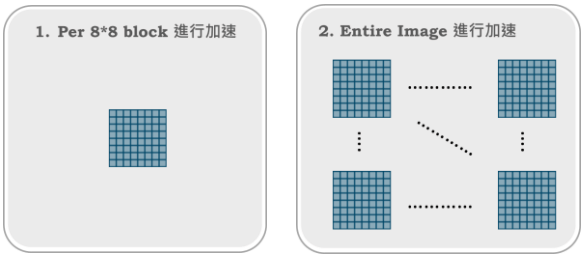


Figure 4: Step-Wise Parallelization Using Pthread and OpenMP.

6.2 CUDA



We try two methods: parallelization per 8*8 block and parallelization on the entire image. In the first method, frequent data transfers between the CPU and GPU introduce significant overhead, leading to poor performance. In contrast, the second method transmits all image pixels at once, minimizing overhead and improving efficiency. However, in the second method, the 8*8 pixel blocks exhibit dependencies in the DCT and IDCT processes. To address this, we utilize shared memory to effectively manage these dependencies and improve performance.

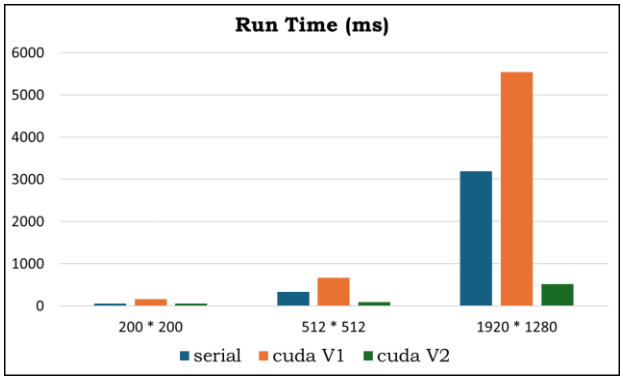


Figure 5: Parallelization Using CUDA.

The chart shows the runtime of three methods (serial, cuda V1, and cuda V2) across different image sizes. For the 200×200 small-sized image, the differences between the methods are minimal. For the 512×512 image, cuda V2 demonstrates better acceleration. However, for the 1920×1280 large-sized image, cuda V1 incurs significant overhead due to excessive data transfers, resulting in the longest runtime, while cuda V2 achieves the shortest runtime, clearly outperforming the other methods. As the resolution increases, cuda V2 shows superior performance, primarily because it optimizes data access using shared memory, reduces global memory transfer overhead, and efficiently allocates computational resources.

6.3 All Models Comparison

Figure 6 shows the speedup performance of three methods (PThread, OpenMP, and CUDA) for different image sizes.

i. 200×200 Small Image:

PThread achieves the highest speedup at approximately 4, followed by OpenMP at around 2.5, while CUDA has the lowest speedup, only about 1. This indicates that for small images, CUDA incurs significant overhead from initialization, preventing it from utilizing its parallelization advantage.

ii. 512×512 Medium Image:

The speedup of all three methods becomes similar. PThread and OpenMP achieve a speedup of around 3, while CUDA slightly exceeds 3. As the image size increases, CUDA starts to leverage its GPU parallel computing advantage, surpassing CPU-based parallel methods.

iii. 1920×1280 Large Image:

CUDA demonstrates a significant speedup of 6, far outperforming PThread (around 4) and OpenMP (around 3.8). This shows that for large images, CUDA fully utilizes the GPU's efficient parallel computation capabilities, while CPU-based methods (PThread and OpenMP) are limited by resource contention and synchronization overhead.

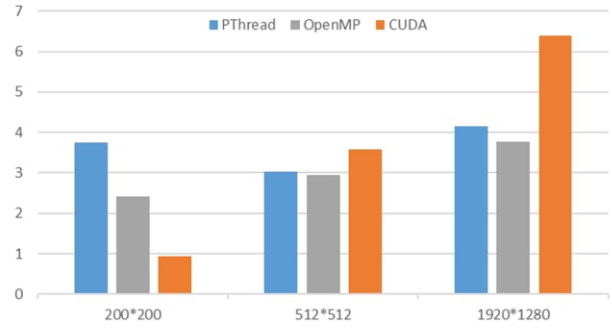


Figure 6: Comparison of three different models.

Overall, PThread performs best for small images, while CUDA shows a significant speedup advantage for large images. As image resolution increases, CUDA's parallel computing capabilities gradually surpass those of PThread and OpenMP.

7 RELATED WORK

JPEG compression is widely used for reducing image file size while maintaining acceptable quality. The process consists of several stages, including color space conversion, block-based transformations, quantization, and entropy encoding. Several studies have explored methods to accelerate these stages through parallel processing on multi-core CPUs and GPUs, focusing on improving efficiency for real-time applications.

D. Liu and X. Y. Fan outlines a structured approach to parallelizing JPEG compression in [1], primarily focusing on multi-core CPUs. They break down the JPEG encoding process into independent tasks, emphasizing two types of parallelism: data parallelism, where different 8x8 pixel blocks are processed in parallel, and pipeline parallelism, where stages like DCT and quantization operate concurrently on different parts of the image. This design takes advantage of the block-based nature of JPEG, with dynamic task scheduling for efficient load balancing across multiple cores. Liu and Fan demonstrate significant speedups, achieving 5x to 7x performance gains on multi-core processors.

On the other hand, [2] addresses the specific stage of color space conversion, a crucial preprocessing step in JPEG compression. Most images are stored in RGB format but need to be converted to a more efficient color space (such as YCbCr) for compression. The study emphasizes the computational intensity of this transformation, particularly for high-resolution images, and proposes using GPUs for acceleration. The authors take advantage of the massive parallelism of GPUs, where each pixel's RGB values are independently converted to YCbCr in parallel, significantly speeding up the process.

8 CONCLUSIONS

This study explored the performance of PThread, OpenMP, and CUDA under different parallelization methods and image sizes. PThread and OpenMP share similar acceleration logic, but PThread requires manual management of thread creation and synchronization, which, if mishandled, may result in inefficient performance improvements. During experiments with different

parallelization methods, we found that data access is a key factor affecting parallelization speed. Frequent access or large-scale data processing can directly impact overall computational performance due to transmission overhead. Additionally, for CUDA, the acceleration performance for smaller images was inferior to other methods and even slower than serial execution. This is likely due to significant time spent on data transfers between the CPU and GPU, which increases the total execution time. However, as image resolution increases, CUDA can fully leverage the GPU's parallel computing capabilities, demonstrating significant acceleration benefits.

REFERENCES

[1] D. Liu and X. Y. Fan, "Parallel program design for JPEG compression encoding," 2012 9th International Conference on Fuzzy Systems and Knowledge Discovery, Chongqing, China, 2012, pp. 2502-2506, doi: 10.1109/FSKD.2012.6234221..

[2] SHEN, Meng; RUAN, Shengdong. Accelerating Color Space Conversion using Graphic Processing Units.